جامعة البليدة 1	**UNIVERSITY OF BLIDA 1** **Computer Science Department**
	4th eng
	TP 3 — Access Control CTF

Your objective is to discover the five flags inside the isolated Docker challenge. This handout intentionally gives you the questions and hints, but not the solutions. For each flag, identify the vulnerability, understand why the access-control model fails, and record the exact flag value and the evidence that proves your discovery.

Important scope rules

- Work only inside the provided challenge container.
- Do not use the root account directly.
- Do not attack the host, Docker socket, or another student's container.
- Do not guess passwords or use kernel exploits.
- Use standard Linux inspection and permission-analysis commands.

FLAG 1

(Hidden does not mean protected)

Challenge question

A sensitive `FLAG{...}` file is hidden somewhere in the lab. Without knowing its path in advance, find the flag and determine why its hidden filename does not prevent your account from accessing it.

Hints

- Start from the lab filesystem rather than guessing a specific directory.
- Think about how Linux can recursively search for files whose names match a pattern.
- Hidden files and directories are still ordinary filesystem objects.
- After finding the file, inspect its permissions and ownership.

What to record

- The exact flag value.
- The complete path to the flag.
- The command(s) that allowed you to discover it.
- The file permissions/ownership.

FLAG 2 *(The confused deputy)*

Challenge question
A privileged helper can access a protected resource on behalf of the caller. Find the SUID program that is relevant to the challenge and use it to obtain the second flag without directly becoming root.

Hints

- Search for files with the SUID permission bit set.
- Inspect suspicious candidates with file metadata and readable strings.
- Run the relevant program first with no arguments and carefully read its usage message.
- Ask yourself: who is the real user, what is the effective identity of the process, and which filename does the caller control?
- The target flag is explicitly located under `/root`, but you should not access it by logging in as root.
- Think about what happens when a privileged program trusts a path supplied by an unprivileged caller.

What to record

- The relevant SUID program.
- The exact command used to retrieve the flag.
- The important output demonstrating the access.
- A short explanation of the missing authorization check.

FLAG 3 *(Sudo and least privilege)*

Challenge question
A root-executed backup mechanism is available through sudo. Determine how the permitted command, the root script, PATH resolution, and an external command can be combined to obtain the third flag.

Hints

- Inspect the sudo policy before executing anything.
- Read the backup script carefully; identify every external command it invokes.
- Pay particular attention to commands invoked without an absolute path.
- Inspect your current PATH and determine where command resolution can search.
- Ask: what exactly is authorized by sudo, and what additional execution chain does that authorization unintentionally permit?
- Do not assume that authorizing a script automatically means the entire execution chain is safe.

What to record

- The relevant sudo rule.
- The vulnerable script/command chain.
- The commands used to obtain the flag.
- The key output proving successful access.

FLAG 4 *(Root executes what you can modify)*

Challenge question
Find a script or executable that is executed with root privileges but can be modified by an unprivileged user. Demonstrate how this permission mismatch can be used to obtain the fourth flag.

Hints

- Inspect running processes and the lab's executable/script directory.
- Read the maintenance script and determine whether it is executed by root.
- Check the owner, group, and permission bits of the relevant file.
- Ask whether your account can modify a file that a privileged process will later execute.
- The important chain is: unprivileged user → writable file → root execution → privileged action → protected resource.
- You are looking for a permission problem, not a password or kernel exploit.

What to record

- The file executed by root.
- Its owner, group, and permissions.
- The commands used to demonstrate the attack.
- The exact fourth flag and the privilege-escalation chain.

FLAG 5 *(Linux capabilities)*

Challenge question
Find a file with a dangerous Linux capability that grants additional privilege or bypasses normal discretionary access controls. Use the capability to obtain the fifth flag.

Hints

- SUID is not the only mechanism that can provide extra privilege.
- Search the filesystem for assigned capabilities.
- Pay special attention to capabilities related to reading protected files or changing process identity.
- Inspect the program associated with the capability and determine what operations it enables.

- Think in terms of: capability → privileged operation → protected resource → flag.

What to record

- The file carrying the capability.
- The capability assigned to it.
- The commands used to obtain the flag.
- The exact fifth flag.
- A short explanation of why the capability is dangerous in this context.

Final check

(Connect the five flags)

After finding all five flags, do not stop at the flag values. For each one, identify the access-control weakness that made the discovery possible.

Flag	Vulnerability	Mechanism	Privilege gained	Protected resource
Flag 1				
Flag 2				
Flag 3				
Flag 4				
Flag 5				

Recommended reasoning pattern: Reconnaissance → Misconfiguration → Extra privilege → Access to a protected resource → Flag

Student evidence checklist

- ☐ I recorded the exact flag value for each challenge.
- ☐ I recorded the discovery path and commands.
- ☐ I preserved the important command output as text.
- ☐ I can explain why the access-control mechanism failed.
- ☐ I can identify the security principle violated by each vulnerability.
- ☐ I can propose a remediation for each weakness.